2025

Алгоритмизация и программирование (1 семестр)

МГТУ им. Н.Э. Баумана

Факультет Информатика и системы управления

Кафедра Компьютерные системы и сети

Лектор: д.т.н., профессор

Иванова Галина Сергеевна

Дисциплина Алгоритмизация и программирование

Объем дисциплины в 1-м семестре – 7 зачетных единиц – 252 час.:

- лекции – 51 час - знакомство с теоретическим материалом;
- семинары – 34 часа - разработка алгоритмов решения задач;
- лабораторные работы – 34 часа – 16*2+2(зачет) часов - изучение приемов программирования;
- самостоятельная работа – 16*6 часов - закрепление материала (6 часов в неделю).

Содержание дисциплины:

Модуль 1 (4 неделя). Основы алгоритмизации и программирование с использованием скалярных типов данных.

Модуль 2 (9 неделя). Структурные типы данных и модульное программирование.

Модуль 3 (12 неделя). Динамические структуры данных и файлы.

Модуль 4 (17 неделя). Основы объектно-ориентированного программирования.

Язык программирования: **C++**
Среда программирования: **Qt Creator**
Операционная система: **ASTRA Linux**

Лабораторные работы:

Место проведения:

Один раз в неделю по 2 часа.

С собой иметь: флешку, материалы лекций или другую литературу, можно свой ноутбук.

Посещение всех занятий обязательно!

Отчетность по дисциплине:

- 4 рубежных контроля (РК):

- РК 1. Итерационные циклы - 2 часа – 3-4 недели.
- РК 2. Матрицы и подпрограммы - 2 часа – 7-8 недели.
- РК 3. Файлы и динамическая память - 2 часа – 10-11 недели.
- РК 4. Иерархии классов - 2 часа – 15-16 недели.

- экзамен.

Экзамен можно сдать досрочно до 20 декабря, если сдать все лабораторные и ДЗ.

Учебные материалы

1. Павловская Т.А. С/С++. Программирование на языке высокого уровня: учебник для вузов. СПб.: Питер, 2021. 464 с.: ил. URL: https://ibooks.ru/bookshelf/376844.
2. Иванова Г.С. Программирование. Учеб. для вузов. – М.: Изд-во Кнорус, 2022. 426 с. – *Только алгоритмизация, так как учебник на языке Паскаль.*
3. Иванова Г.С., Ничушкина Т.Н. Объектно-ориентированное программирование. Учеб. для вузов. – М.: Изд-во МГТУ им. Н.Э. Баумана, 2014. 456 с.
4. Иванова Г.С., Ничушкина Т.Н., Самарев Р.С. . С++. Часть 1. Средства процедурного программирования Microsoft Visual C++ 2008: Учебное пособие. – М.: МГТУ им. Н.Э. Баумана, 2010. – 126 с. – Эл.уч.изд.
5. Иванова Г.С., Ничушкина Т.Н. С++. Часть 2. Объектно-ориентированное программирование на языке С++ в среде Visual Studio 2008: Учебное пособие. – М.: МГТУ им. Н.Э. Баумана, 2013. – 161 с. – Эл.уч.изд.
6. Шилдт Г. Полный справочник по С++, 4 изд. М.: Изд. дом "Вильямс", 2015. 800с.
7. Сайт кафедры: **https://e-learning.bmstu.ru/iu6/course/view.php?id=366.**

Консультации д.т.н., проф. Г.С. Ивановой

а) на кафедре ИУ6 (главное здание, 8 этаж, ауд. 807):

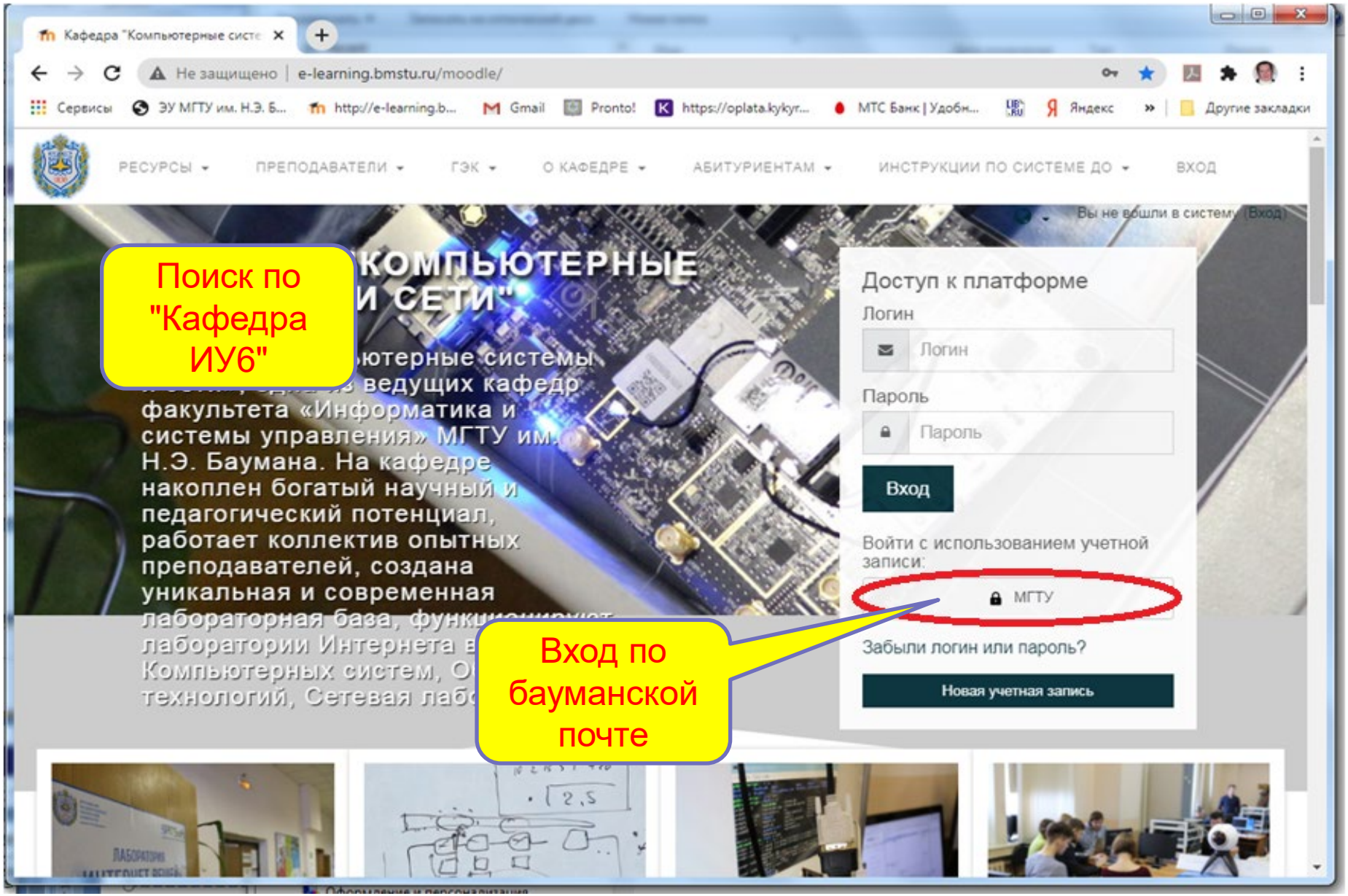
- понедельник с 11-00 до 12-00 и с 13-00 до 15-00;
- вторник с 14-00

б) по электронной почте:

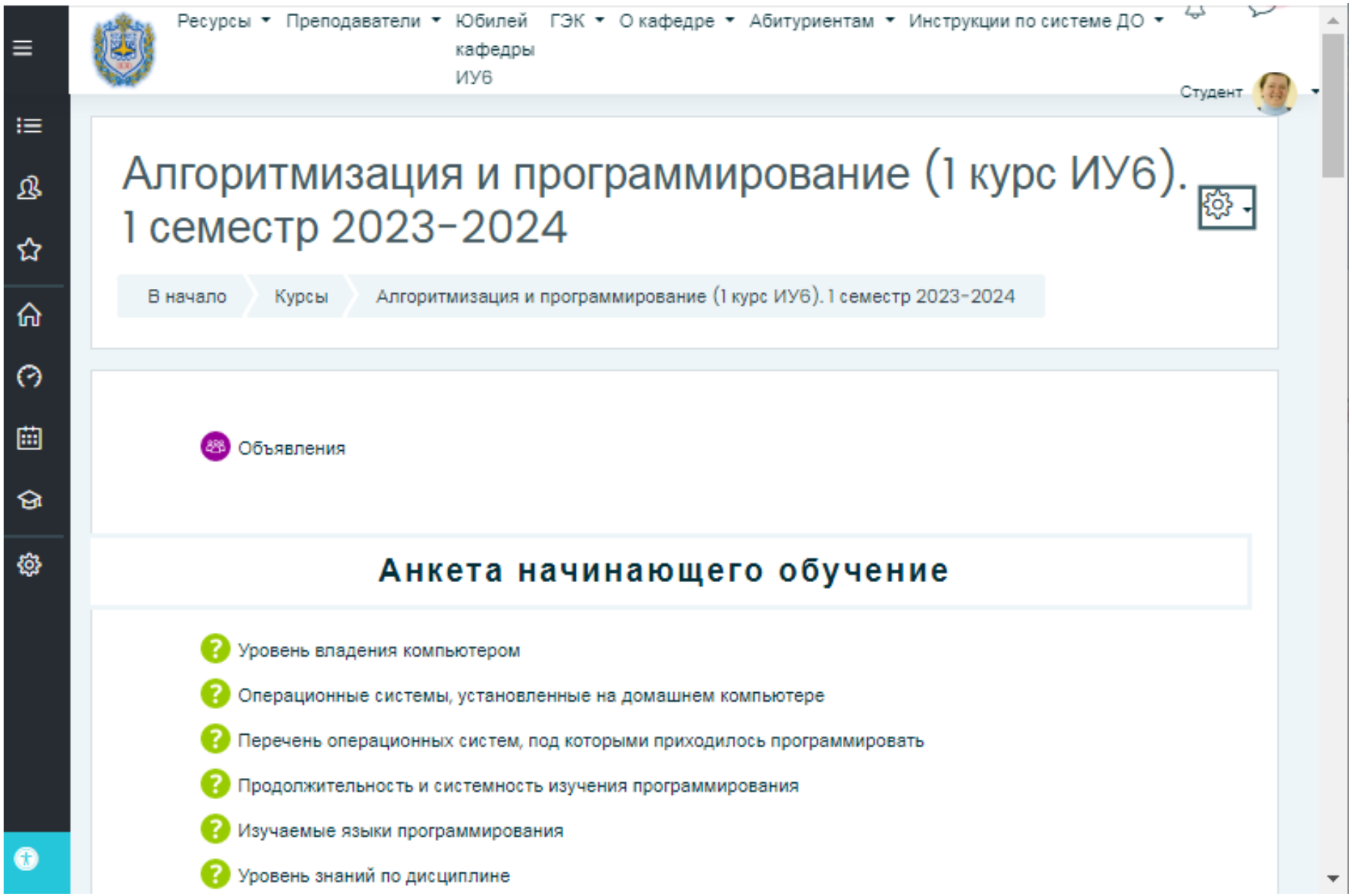
gsivanova@gmail.com,
gsivanova@bmstu.ru

в) возможны удаленные консультации.

Вход на сайт кафедры



Страница дисциплины. Анкета



Учебно-технологическая практика (ИУ6)

Согласно приказу ректора во все планы бакалавров в качестве обычной или факультативной дисциплин внесены учебно-технологические практики:

- на каф. ИУ6 – сборка-разборка компьютера;
- на каф. РК6 – пайка.

На каф. ИУ6 практика для групп ИУ6-13Б, ИУ6-14Б, ИУ6-15Б начнется **13 сентября**. Группы ИУ6-11Б, ИУ6-12Б эту практику будут проходить весной.

Практика будет проходить по субботам в составе ½ группы (ауд. 804):

- гр. ИУ6-1…Б – с 8-30 до 11-40;
- гр. ИУ6-1…Б – с 12-25 до 15-35;
- гр. ИУ6-1…Б – с 15-55 до 19-05.

Конкретное расписание будет согласовано со старостами и объявлено позднее.

Часть 1. Основы алгоритмизации и процедурное программирование

МГТУ им. Н.Э. Баумана
Факультет Информатика и системы управления
Кафедра Компьютерные системы и сети
Лектор: д.т.н., проф.
Иванова Галина Сергеевна

Введение

C++ – универсальный язык программирования высокого уровня. Поддерживает структурный и объектный подходы, а также концепцию обобщенного программирования. Построен на базе языка Си. Первоначально имел название "С с классами".

Автор языка: Бьярне Страуструп.

Год выхода: 1983 г.

Достоинства языка:

- универсальность;
- компактность записи;
- высокая производительность;
- наличие большого количества библиотек.

Недостатки:

- незащищенный синтаксис, не позволяющий компилятору автоматически выявлять значительное количество синтаксических ошибок программы;
- сложная семантика используемых конструкций, не позволяющая легко читать программу = > большая вероятность ошибок.

Среды программирования

Среда программирования – собранная в единую программную систему совокупность программных средств, предназначенный для разработки программных продуктов. Обычно включает: редактор текстов, компилятор языка программирования, компоновщик, отладчик, библиотеки подпрограмм и/или классов, средства профилирования и т.п.

Основные среды для разработки программ на языке C++:

- среда программирования ***Visual Studio*** крупнейшей международной IT-компании Microsoft. Продукт ***Visual Studio Community*** – бесплатная некоммерческая профессиональная версия среды разработки программ под Windows;

- среда программирования ***Qt Creator*** – бесплатная профессиональная кросплатформенная среда, в настоящее время принадлежит QT Company.

Этапы создания ПО

1. **Постановка задачи** – неформальное описание задачи.

2. **Анализ и уточнение требований** – формальная постановка задачи и выбор метода решения.

3. **Проектирование** – разработка структуры программного продукта, выбор структур данных, выбор метода решения, разработка алгоритмов обработки данных, определение особенностей взаимодействия с программной средой и т.п.

4. **Реализация** – составление программ, их тестирование и отладка.

5. **Модификация** – выпуск новых версий.

Пример разработки программы

1. **Постановка задачи**: Разработать программу, которая определяет наибольший общий делитель (НОД) двух целых чисел.

2. **Анализ и уточнение требований:**

   1) *Функциональные требования*

      исходные данные: a, b – натуральные числа; 0 < a, b < **?** ;

      результат: x – натуральное число, такое, что

$$\mathbf{x = \max\{y_i \ / \ i = \overline{1,n}\}, \text{ где } ((a \bmod y_i) = 0) \ \& \ (b \bmod y_i) = 0)}$$

   Методы решения:

      а) найти делители $Y = \{ y_i \}$ и определить $x = \max\{Y\}$;

      б) метод Евклида

Пример 1:

a	b
24	18
6	18
6	12
6 =	6

Пример 2:

a	b
3	4
3	1
2	1
1 =	1

Пример разработки программы (2)

2) *Эксплуатационные требования*:

а) операционная система – Linux или Windows (консольный режим);

б) процессор – не ниже Pentium;

в) предусмотреть запрос на ввод данных с клавиатуры;

г) результаты вывести на экран.

3) *Технологические требования*:

а) язык программирования: C++;

б) среда программирования: Qt Creator;

в) технология программирования: структурный подход.

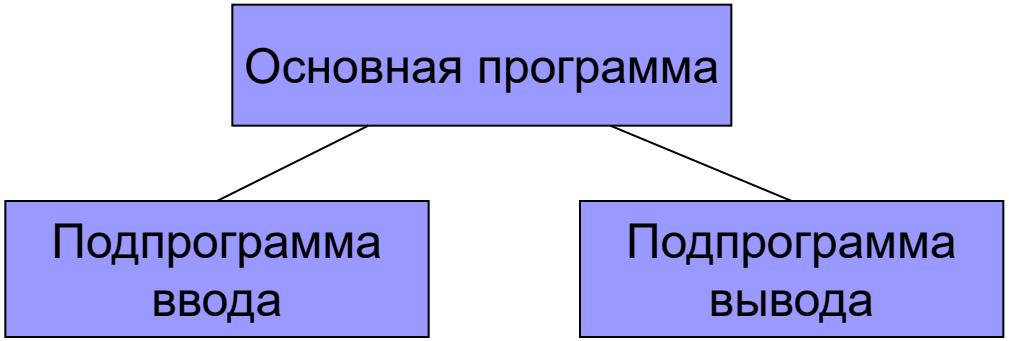
Следует иметь в виду, что технологические требования часто не задают. Обычно они устанавливаются разработчиками программ.

Пример разработки программы(3)

3. Проектирование

Виды проектной документации:

1 ***Структурная схема ПО*** **–** показывает взаимодействие по управлению основной программы и подпрограмм.



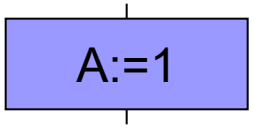
2 ***Схемы алгоритмов программы и подпрограмм***

Схемы алгоритмов

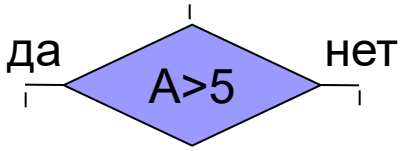
Обозначения по ГОСТ 19.701 – 90

1. Терминатор (начало/конец)

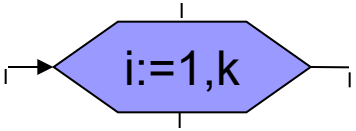
2. Процесс (вычисление)



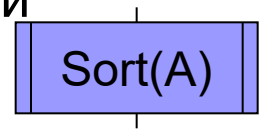
3. Анализ (проверка)



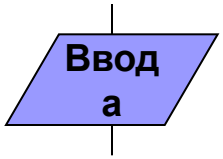
4. Модификатор (автоматическое изменение)



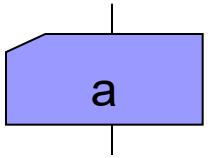
5. Предопределенный процесс (подпрограмма)



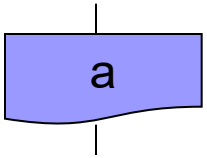
6. Ввод/вывод данных



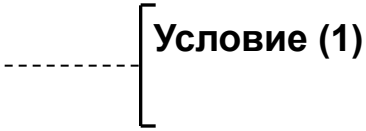
7. Ввод с перфокарт



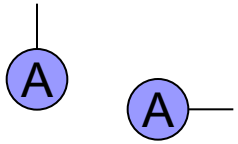
8. Вывод на принтер



9. Комментарий



10. Соединитель



Правила выполнения схем алгоритмов

- Схемы алгоритмов должны быть выполнены аккуратно, желательно с применением графических редакторов на компьютере.
- Стрелки на линиях, идущих **сверху вниз** и **слева направо**, т. е. в направлении письма, **не ставят**, чтобы не затенять схему.
- Если линия – ломанная, и направление ее хотя бы в одном сегменте не совпадает со стандартными, то стрелка ставится только **в конце линии**, перед блоком, в который она входит.
- Если схема не умещается на странице или линии многократно пересекаются, то линии разрывают. Один соединитель ставится в месте разрыва, второй – в месте продолжения линии. Оба соединителя помечаются одной и той же буквой или цифрой.
- Для простоты чтения схемы ее начало должно быть сверху, а конец – снизу. При этом количество изгибов, пересечений и обратных направлений соединительных линий должно быть **минимальным**.

Пример неудачного изображение схемы

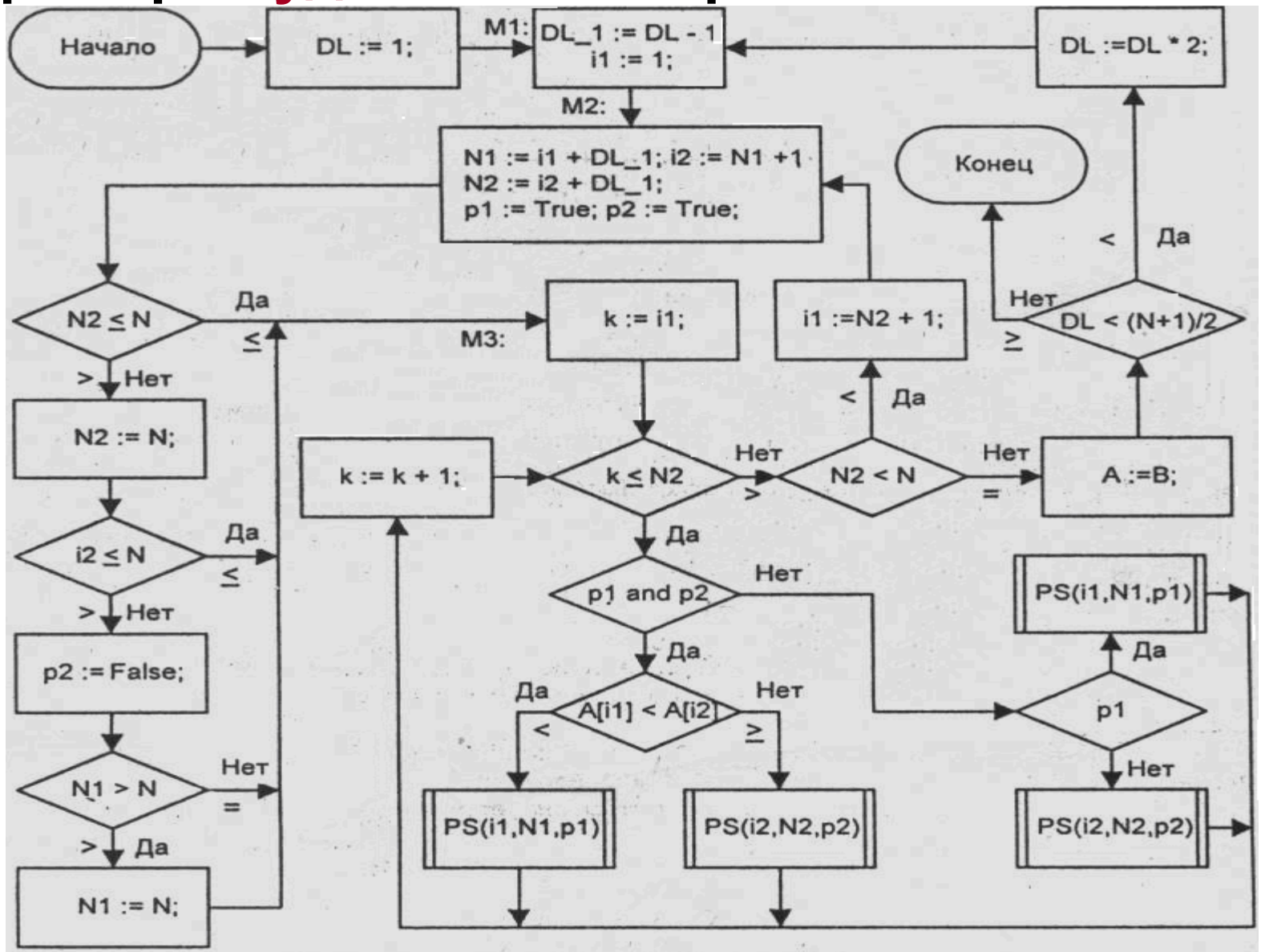
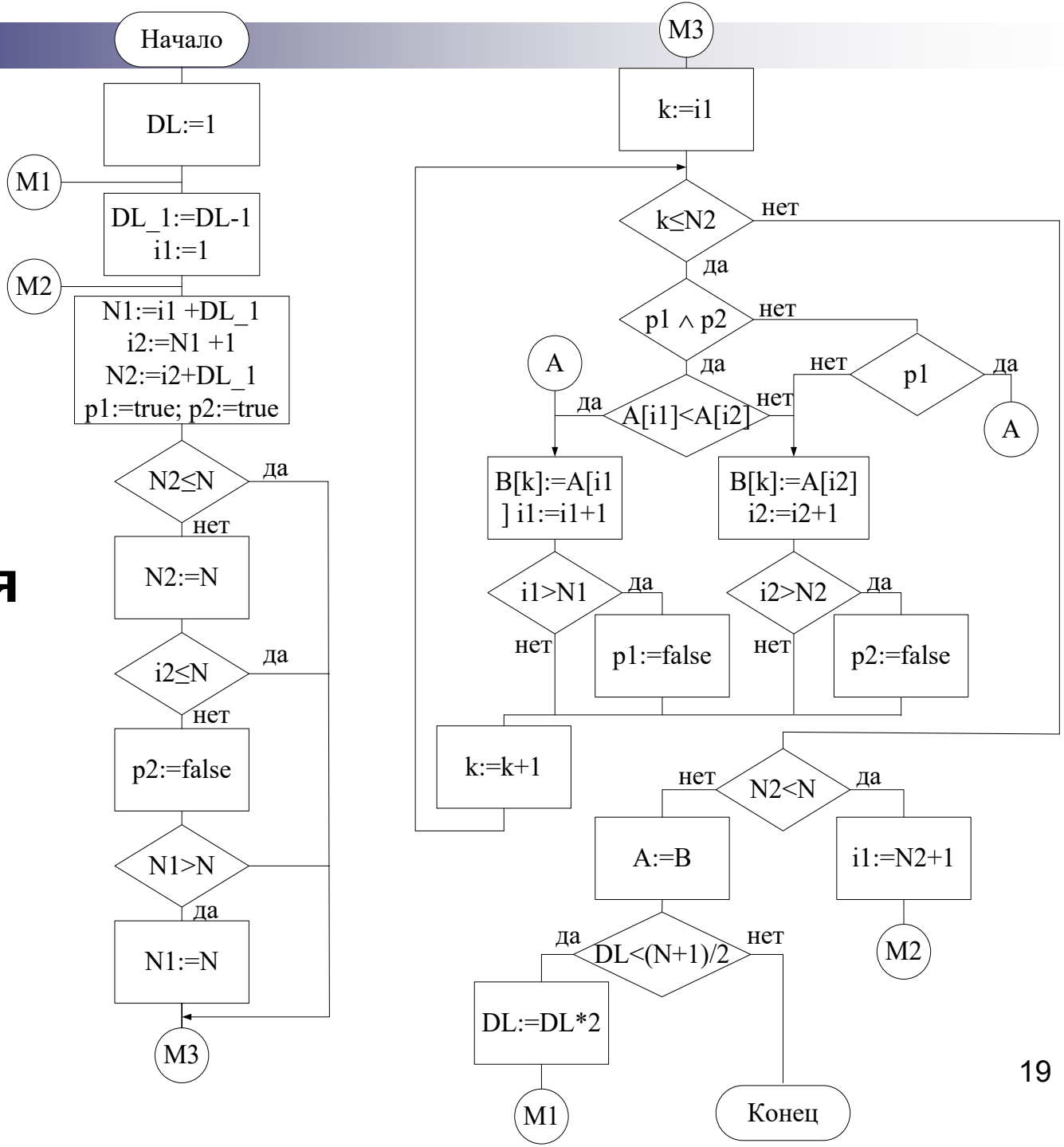
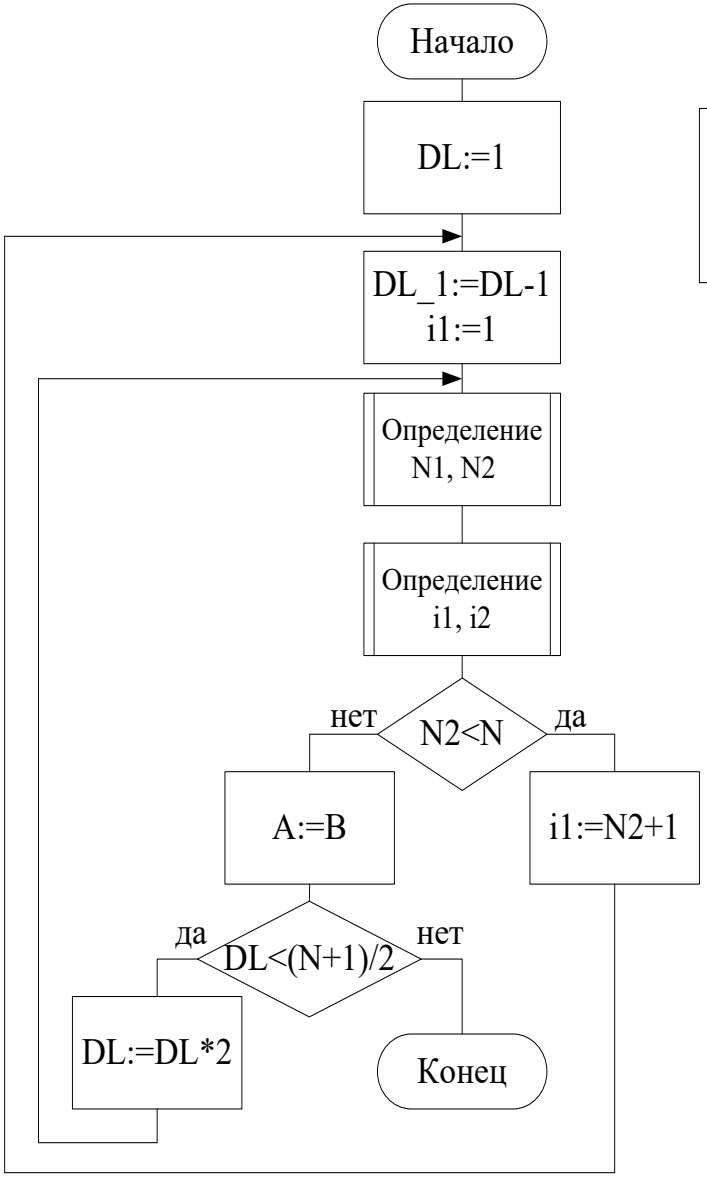


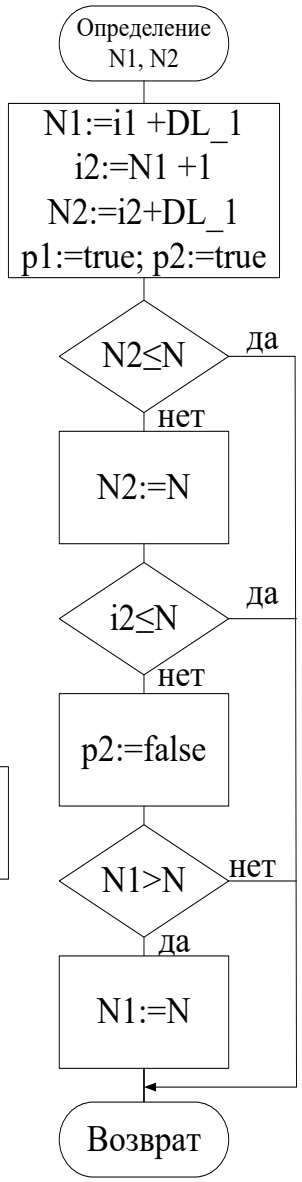
Рис. 14.18
Схема алгоритма сортировки последовательными слияниями

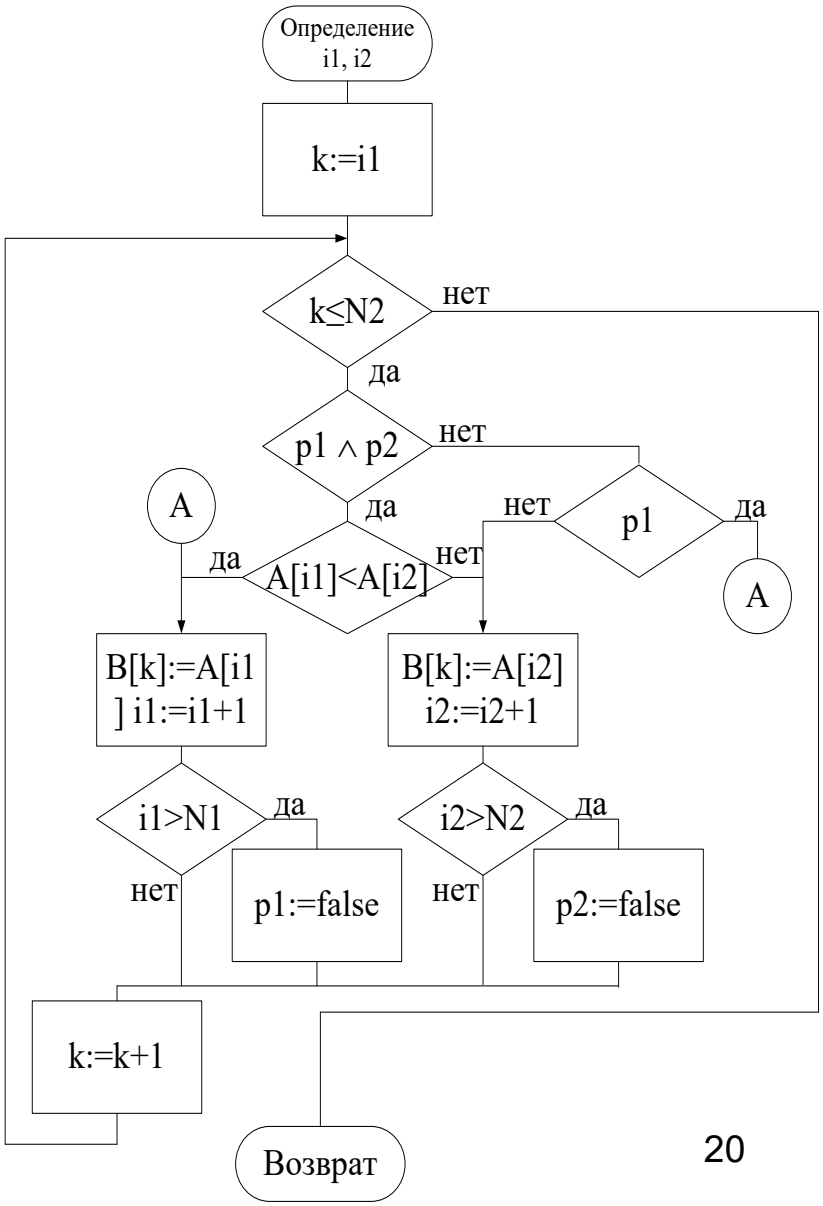
1-й вариант более читаемого изображения схемы алгоритма



2-й вариант: выделение подпрограмм

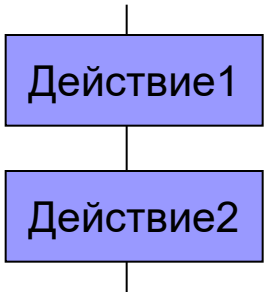




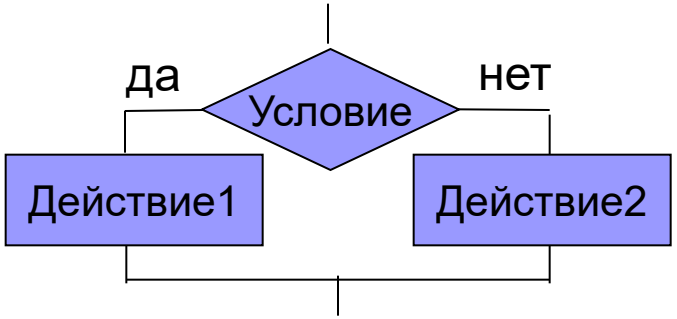


Основные структурные конструкции алгоритма

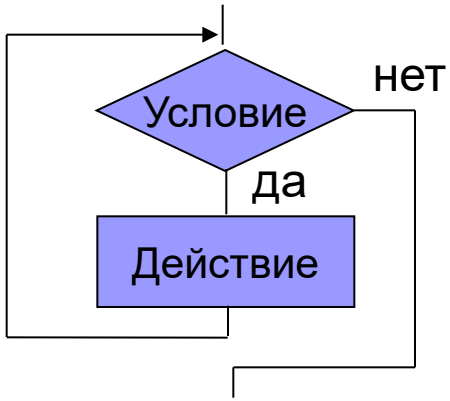
1. Следование



2. Ветвление



3. Цикл-пока



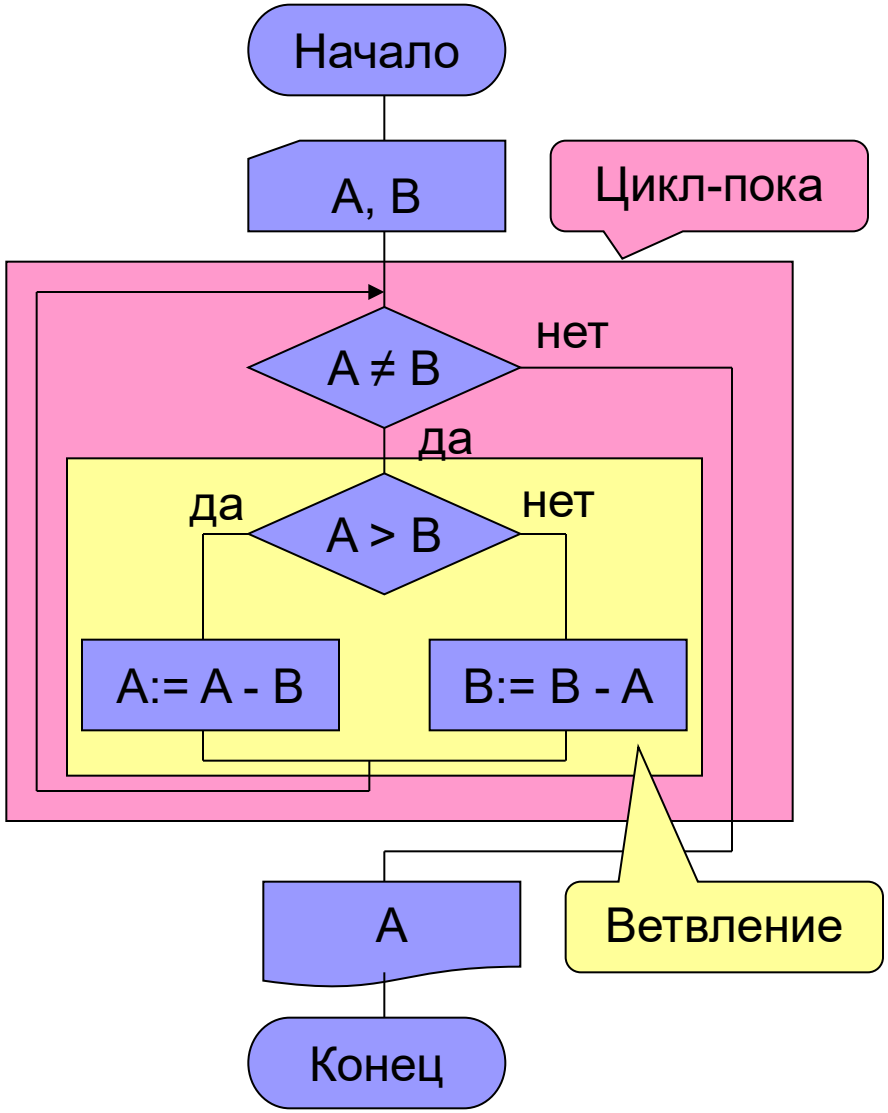
Псевдокод:

…
Действие 1
Действие 2
…

…
Если Условие
то Действие 1
иначе Действие 2
Все-если
…

…
Цикл-пока Условие
Действие
Все-цикл
…

Схема и псевдокод алгоритма программы поиска НОД



Алгоритм Евклида:

Ввести A,B

Цикл-пока A ≠ B

Если A > B

то A := A – B

иначе B := B – A

Все-если

Все-цикл

Вывести A

Конец

4. **Реализация программы, ее тестирование и отладка.**

Простая консольная программа (Ex01_01)

Программа – последовательность инструкций, адресованных компьютеру, которая точно определяет, как следует решать задачу.

```
#include <iostream>
using namespace std;

int main()
{
    int a, b;
    cout << "Enter a, b\n";
    cin >> a >> b;

    while (a!=b)
      if  (a>b) a=a-b;
      else b=b-a;

    cout <<"nod = "<<a<<'\n';

    return 0;
}
```

Подключение библиотеки

Подключение пространства имен стандартных библиотек

Главная функция – основная программа

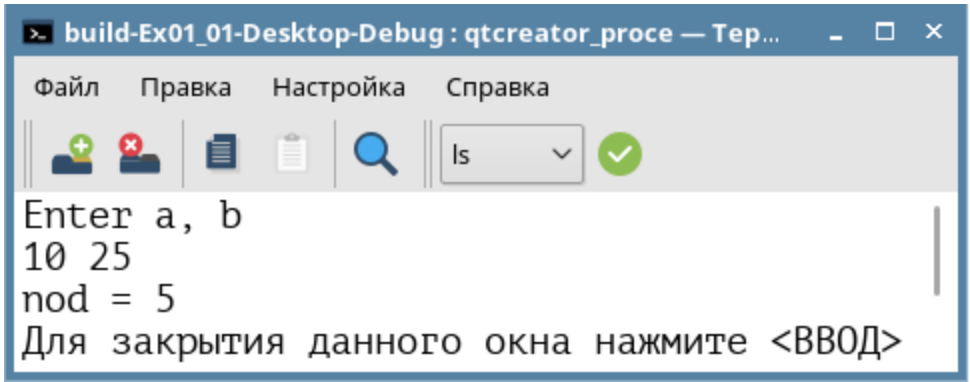
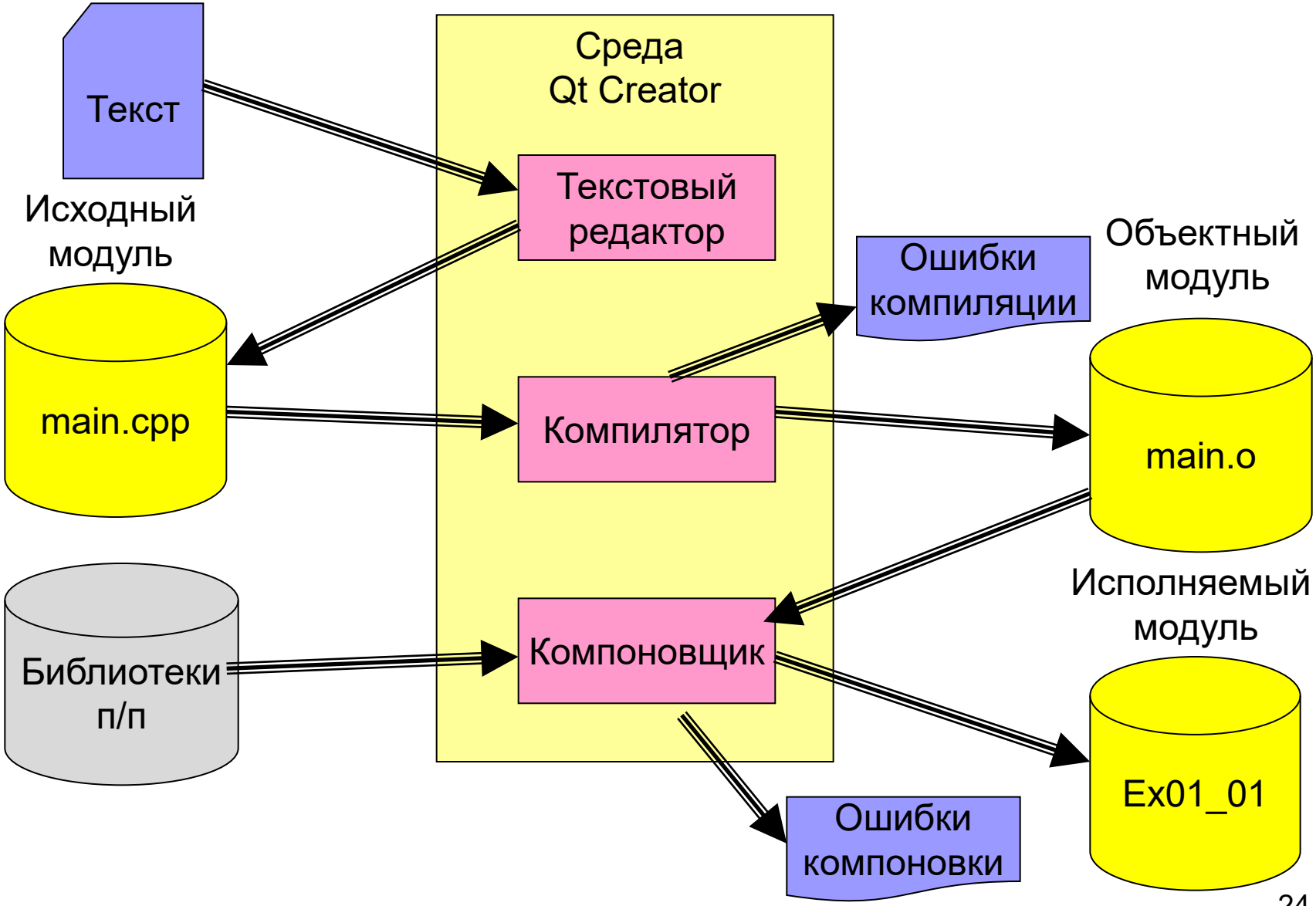
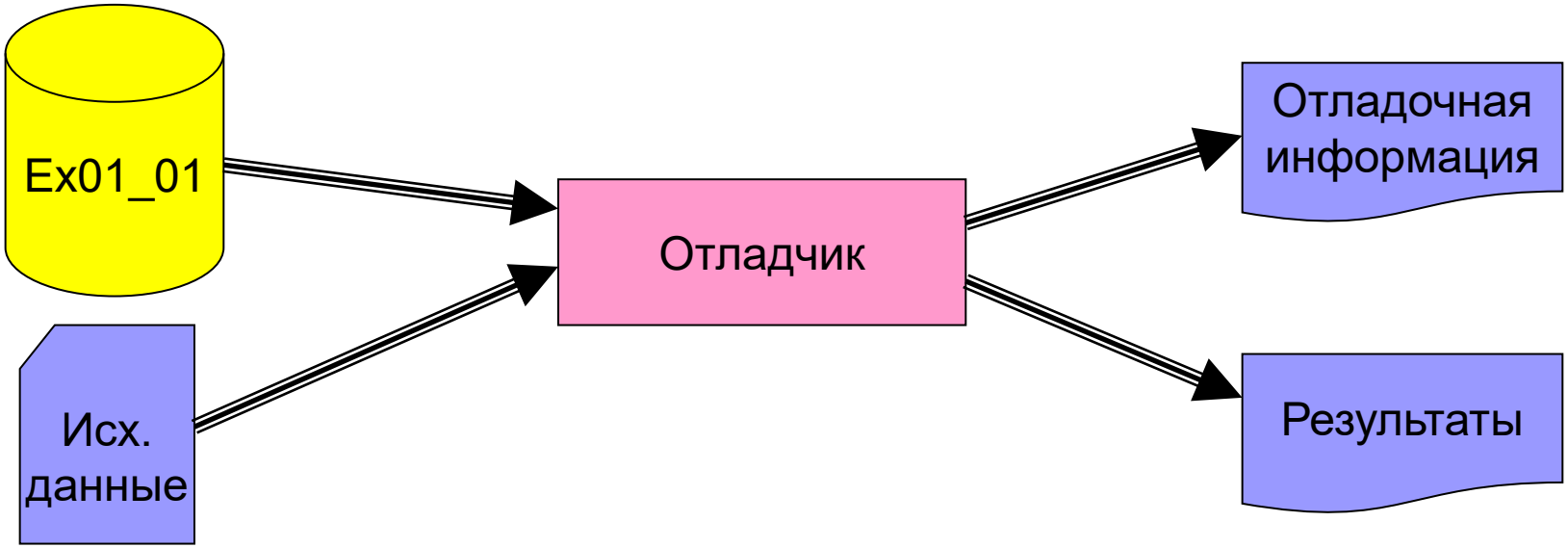
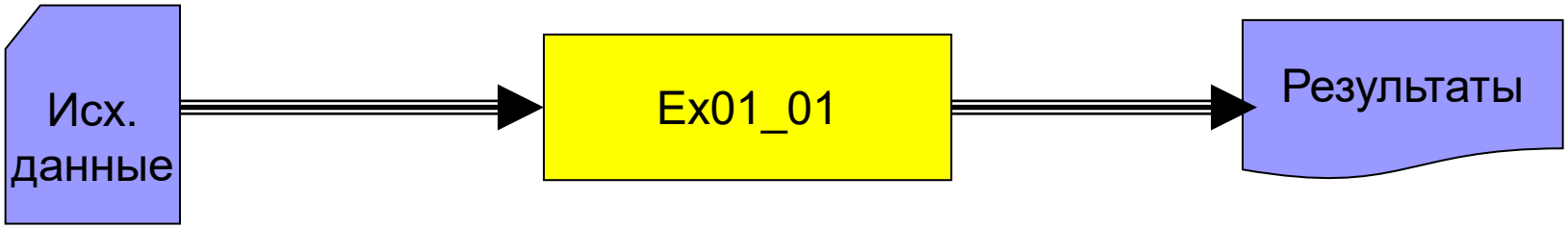


Схема процесса подготовки программы



Схемы процессов отладки и выполнения программы





Глава 1 Простейшие конструкции языка C++

1.1 Синтаксис языка C++

! Строчные и прописные буквы различаются!

Алфавит языка C++ включает:

- строчные и прописные латинские буквы: `_, a, A, b, B` и т.д.;
- арабские цифры: `0, 1, 2, 3, 4, 5, 6, 7, 8, 9`;
- шестнадцатеричные цифры: `0..9, a..f` или `A..F`;
- специальные символы: `+ - * / = ; { }` и т. д.;
- служебные слова: `do, while, for, if, else` и т. д.
- прочие символы: `пробел ' ', символ табуляции '\t'`.

Из символов алфавита формируются лексемы ("слова" языка).

Лексема – единица текста программы, расположенная между разделителями. Разделителями могут служить: специальные символы, пробелы и символы табуляции.

Лексемами C++ являются:

- *идентификаторы* – имена программных элементов (переменных, именованных констант, функций и т.п.);
- *литералы* – данные, непосредственно указанные в программе;
- *специальные символы* и *служебные слова*.

Идентификаторы. Описание синтаксиса

Идентификатор – последовательность из букв латинского алфавита (включая символ "_") и десятичных цифр, начинающаяся с буквы.

- Синтаксис в форме Бэкуса-Наура (БНФ):

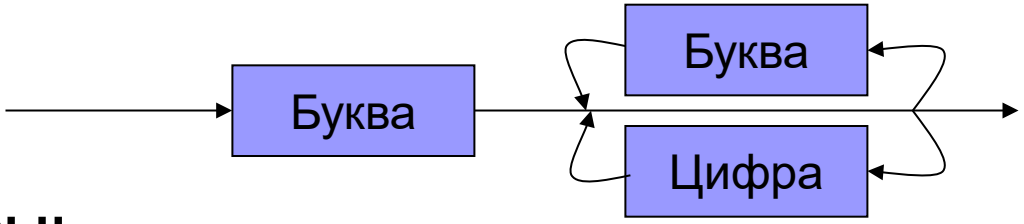
<Идентификатор> ::= <Буква> | <Идентификатор><Буква> |

<Идентификатор><Цифра>

- Синтаксис в расширенной форме Бэкуса-Наура (РБНФ):

$ Идентификатор = Буква {Буква | Цифра}

- Синтаксис задан синтаксической диаграммой:



Компиляторы задают ограничение на длину идентификатора, например, для VS C++ - 2048 символов

Примеры:

Правильные идентификаторы: A, a21, n1dw, kk_5

Неправильные идентификаторы: 12sdd, ?hjj, s21*5

Обозначения расширенной формы Бэкуса-Наура:

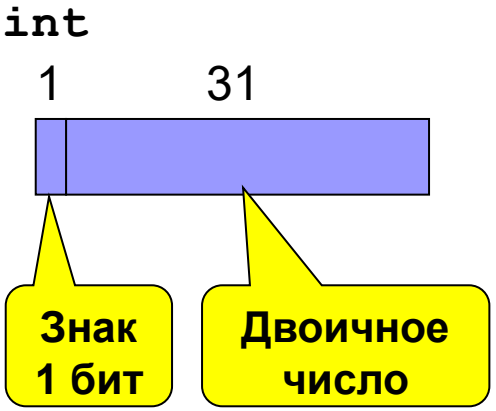
- ***нетерминальные символы (основные понятия)*** - обозначаются словами, написанными на русском языке; если нетерминал состоит из нескольких смысловых слов, то они должны быть написаны слитно (можно слова разделять подчеркиванием или начинать с прописной буквы);
- ***терминальные символы (символы алфавита)*** - обозначаются символами алфавита языка;
- **$** - начало синтаксического правила;
- **=** (равно) - отделяет левую часть правила от правой;
- **[и]** - означает, что заключенная в скобки синтаксическая конструкция может отсутствовать;
- **{ и }** - означает, что заключенная в скобки синтаксическая конструкция повторяется *0..n* раз;
- **|** - используется для обозначения альтернатив;
- **(и)** - используется для ограничения последовательности альтернативных конструкций.

Красным показаны символы, относящиеся к записи РБНФ

Пример: $ Идентификатор = Буква {Буква | Цифра}

1.2 Представление данных в компьютере

Компьютер "умеет" обрабатывать данные - целые и вещественные числа, представленные *в двоичной системе счисления*.



Целые числа в компьютере могут храниться со знаком и без. Если используется беззнаковый формат, то числа считаются положительными. При хранении со знаком целые числа могут быть и положительными и отрицательными. Для хранения знака используется один двоичный разряд (старший).

Целые числа в компьютере представляются точно.

Размер числа определяется типом.

Пример:

а) `int a = 45;`

$45_{10} = 2^5 + 2^3 + 2^2 + 2^0 = 10111_2$

a = 0 0000000 00000000 00000000 00010111

б) `int b = -45;`

b = 1 1111111 11111111 11111111 11101001

Отрицательные целые числа хранятся в *дополнительном* коде

Основные встроенные (фундаментальные) целочисленные типы данных:

Имя типа	Подтипы	Размер, байт	Значения
int или **long**	**[signed] int** **unsigned int**	4	$-2^{31}..\ 2^{31}-1$ $0..\ 2^{32}-1$
short	**[signed] short** **unsigned short**	2	-32768..32767 0..65535
long long	**[signed] long long** **unsigned long long**	8	$-2^{63}..\ 2^{63}-1$ $0..\ 2^{64}-1$
char	**[signed] char** **unsigned char**	1	-128..127 0..255
wchar_t		2	0..65535
bool		1	false (0), true(1)

Для совместимости с Си по-прежнему считается: 0 – false; не 0 – true.

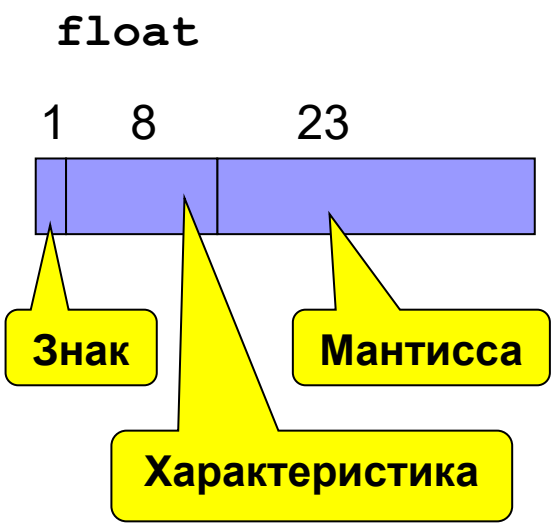
Вещественные числа представляются в компьютере в формате с плавающей точкой (в экспоненциальной форме):

$$\pm\text{Мантисса}*2^{\pm\text{Порядок}}$$

Однако в памяти хранится не *порядок*, а *характеристика*:

Характеристика = Порядок + Константа_типа

Характеристика всегда положительна.



Пример:

```
float f = 2.625;
```

$2.625_{10} = 10.101_2 = 1.0101_2 * 2^1$

Нормализованное представление (1 – не хранится)

Мантисса = 0101_2

Характеристика = $1_2 + \mathbf{127}_{10} = 1_2 + 1111111_2 = 10000000_2$

f: 0 10000000 0101000 00000000 00000000

⚠ В памяти вещественные числа хранятся с точностью, ограниченной разрядной сеткой.

Конкретная разрядная сетка определяется *типом* данных.

Основные встроенные вещественные типы

Тип	Размер, байтов/ битов	Значащих ("точных") десятичных цифр в мантиссе	Минимальное положительное число	Максимальное положительное число
float	4/32	6	1.175494351e-38	3.402823466e38
double	8/64	15	2.2250738585072014 e-308	1.797693134862318 e308
long double	16/128	18		1.1897314e4932

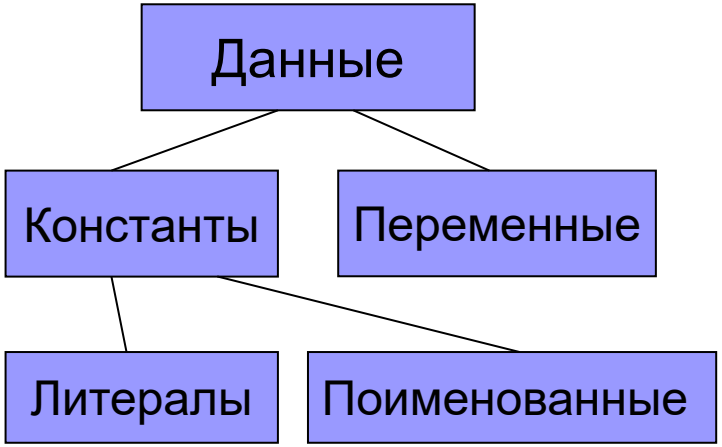
Дополнительный встроенный тип

Неопределенный («пустой») тип void

Нельзя объявлять значения типа void, этот тип используется только при объявлении:

- нетипизированных указателей;
- функций, не возвращающих значений (процедур).

Константы и переменные



- **Константы** – данные, не изменяемые при выполнении программы:
  - *литералы* – константы, указанные непосредственно в тексте программы;
  - *поименованные константы* – константы, обращение к которым выполняется по предварительно объявленным именам.
- **Переменные –** поименованные данные, которые могут меняться при выполнении программы.

Примеры литералов:

а) `-25, 2.5, 0.1e6 /*= 0,1·10⁶*/` – десятичные литералы;

б) `0x2a` – шестнадцатеричный литерал;

в) `true(1), false(0)` – логические литералы;

г) `'d', 'A'` – символьные литералы;

д) `"abcd"` – строковый литерал.

Перечисляемый тип

Используется для объявления совокупности поименованных констант.

$ Перечисляемый_тип =

enum [class Имя] { Идентификатор[= Целое]

{, Идентификатор[= Целое]}}

[Список_идентификаторов];

Красным показаны символы, относящиеся к обозначениям РБНФ

Примеры:

```
enum {SUN, MON, TUES, FRI=5, SAT} day;
```

SUN =0, MON = 1, TUES = 2, FRI=5, SAT=6

Имя переменной

Задание типа позволяет ограничить область видимости перечисленных значений (C++ 11), например:

```
enum class Options {None, One, All};
Options o = Options::All;
```

Объявление типа данных

При необходимости программист может объявить свои типы данных. Возможность появилась только в C++.

$ Тип_данных =

typedef Описание_типа Имя_типа;

Примеры:

Имя нового типа

а) `typedef unsigned int word;`

б) `typedef enum {false, true} boolean;`

Имя нового типа

Объявление переменных и поименованных констант

$ **Объявление_поименованных_данных =**

[Изменчивость] Тип

Идентификатор(=Литерал|(Литерал)|{Литерал})

{,Идентификатор(=Литерал|(Литерал)|{Литерал})};

где Изменчивость – описатель возможности изменения значений:

`const` – константа – неизменяемое значение,

`volatile` – независимо меняющаяся переменная,

без указания изменчивости – обычная переменная;

Тип – описатель типа: `int`, `char`, `float`, `double` и т.д.;

Идентификатор – имя переменной или константы;

Значение – значение переменной (инициализирующее) или константы.

Примеры:

а) `int a,b;` // две целочисленные переменные

б) `float c=1.05,d(3.5),e{1.1};` // инициализированные переменные

в) `const unsigned char letter='a';` // константа – код буквы «а»

г) `int const a=15;` // целочисленная константа 15

д) `const int a(1);` // целочисленная константа 1

1.3 Операции

В процессе написания программы над данными выполняют операции.

Перечень выполняемых операций определяется типом данных.

Арифметические операции выполняются над целыми и вещественными числами (константами и переменными):

+ – сложение; **-** – вычитание; * – умножение;

/ – деление:

- ✓ результат – вещественное, если хотя бы одно из чисел – вещественное,
- ✓ результат – целое, если делимое и делитель – целые,

% - остаток от деления целых чисел.

Примеры:

а)
```
int a = 3, b = -5;
a + b      // -2
a / b      //  0
a % b      // -3 – знак зависит от реализации!
```

б)
```
float g = 3.5,h = -8.1;
g / h      // примерно -0.432
```

в)
```
a / g      // примерно 0.857
```

Логические операции

Логические операции выполняются над логическими константами и переменными:

! (НЕ), && (И), || (ИЛИ)

! (НЕ)

false	true
true	false

&& (И)

	false	true
false	false	false
true	false	true

|| (ИЛИ)

	false	true
false	false	true
true	true	true

Примеры:

```
bool a = true, b = false;
a && b   // false
a || b   // true
```

Поразрядные операции

Поразрядные операции выполняются над каждым разрядом числа (чисел), применяют к целым числам, результат – целое число:

~ (не), & (и), |(или), ^ (исключающее или) ,

>> (сдвиг вправо), << (сдвиг влево)

Примеры:

```
short a = 5;
    ~a
// поразрядное НЕ
```

00000000 00000101 ⇒ 11111111 11111010

5_{10}

-6_{10} или 65530_{10}

```
   a << 2
// сдвиг влево
на два разряда
```

00000000 00000101 ⇒ 00000000 00010100

5_{10}

20_{10}

Операции отношения

Операции отношения выполняются над целыми и вещественными числами и символами. Результат операции true или false:

< (меньше), **>** (больше), **<=** (меньше или равно),

>= (больше или равно), **==** (равно), **! =** (неравно).

Примеры:

1) `int a = 5, b = 13;`

   `a < b` // true

2) `char ch = 'A', cg = 'G';`

   `ch > cg` // false

Примечание. Поскольку вещественные числа в памяти компьютера представляются не точно, при проверке равенства или неравенства вещественных чисел следует задавать точность, например:

3) `float f,q;`

   `fabs(f-q) > 1e-6` // если true, то числа не равны

   `fabs(f-q) <= 1e-6` // если true, то числа равны

Операции порядковые

Порядковые операции выполняются над целыми числами:

Следующее ++Операнд, Операнд++ - увеличивает операнд на 1

Предыдущее -- Операнд, Операнд -- - уменьшает операнд на 1

Если знак операции стоит перед операндом, то в выражении сначала выполняется операция и уже измененный операнд участвует в других операциях. Если знак операции стоит после операнда, то сначала операнд участвует в других операциях, а потом уже изменяется.

Пример:

```
int a=3, b=8;
++a;    // a = 4
b--;    // в = 7
```

Примечание. Если позволяет семантика операции, то добавление символа ";" превращает запись операции или выражение, в законченное предложение языка - *инструкцию* или *оператор*.

Операции присваивания

Операции присваивания выполняются над целыми и вещественными числами, логическими значениями и символами. В качестве левого операнда может использоваться только переменная. Операции:

= - присваивает левому операнду результат вычисления выражения, записанного справа;

+= -= *= /= %= &= ^= |= <<= >>= - присваивают левому операнду результат указанной с символом равно операции над левым и правым операндами.

Примеры:

```
a = 5;    // переменной a присвоено значение 5
a += 2;   // эквивалентно a = a + 2; - переменной a присвоено
          // значение 7
```

Условная операция

Условная операция реализует упрощенный вариант ветвления:

Выражение1 ? Выражение2 : Выражение3

Если результат Выражения1 = true, то результат операции равен результату Выражения2, иначе – результату Выражения3.

Примеры:

```
a = x < 0 ? -x : x;

printf("%3d%c%",a,i == n?' ':'\n'); // результат операции –
                                    // пробел или переход на следующую строку
```

1.4 Выражение

Выражение – это совокупность операндов, объединенных знаками операций. Операндами могут служить константы, переменные и явные вызовы функций.

В процессе выполнения программы выражения *вычисляются*, т.е. над операндами в порядке, определенном приоритетами, выполняются используемые в выражении операции. Вычисления продолжаются до тех пор, пока не останется единственное значение – *результат выражения*.

Если семантика выражения позволяет, то выражение может быть преобразовано в оператор.

Примеры:

1) `(a + b) / c` // выражение преобразовывать в оператор
// бессмысленно: результат вычисления не сохраняется

2) `t = (a + b) / c;` // оператор: результат будет записан в
// переменную t

Математические функции (библиотека <cmath>):

fabs(Вещественное_выражение) // абсолютное значение
abs(Целочисленное_выражение) // абсолютное значение
sqrt(Вещественное_выражение) // $\sqrt{x}$
exp(Вещественное_выражение) // e^x
log(Вещественное_выражение) // ln x
log10 (Вещественное_выражение) // $\log_{10}(x)$
sin(Вещественное_выражение)
cos(Вещественное_выражение)
atan(Вещественное_выражение) // arctg x
tan(Вещественное_выражение) // tg x
acos (Вещественное выражение) // арккосинус
asin (Вещественное выражение) // арксинус
sinh(Вещественное выражение) // гиперболический синус
cosh(Вещественное выражение) // гиперболический косинус

Библиотека **<random>**
rand () // генерация случайного числа $0 \le x < 2^{15}-1$
srand (Ц. выр.) // инициализация генератора случайных чисел

Приоритет операций, начиная с максимального

1. **( ) [ ] -> :: .**
2. **!** (не) **+ - ++ -- &(**адрес**) *(**указатель**) sizeof new delete**
3. **.* ->***
4. *** / %**
5. **+ -** (бинарные)
6. **<< >>**
7. **< <= > >=**
8. **= = ! =**
9. **&**(поразрядное и)
10. **^**(исключающее или)
11. **|** (поразрядное или)
12. **&&**
13. **||**
14. **?:**
15. **= *= /= %= += -= &= ^= |= <<= >>=**
16. **,**

Составное выражение

$ Составное_выражение = Выражение{,Выражение>}

Результатом составного выражения считается результат последнего выражения, вошедшего в составное.

Примеры выражений:

а) `int a=10, b=3; float ret; ret=a/b;` — ret=3

б) `c=1; b=c++;` — b=1, c=2

в) `c=1; sum=++c;` — c=2, sum=2

г) `c = a << 4;` — эквивалентно **c=a*16;**

д) `a+=b;` — эквивалентно **a=a+b;**

е) `a=b=5;` — эквивалентно **b=5; a=b;**

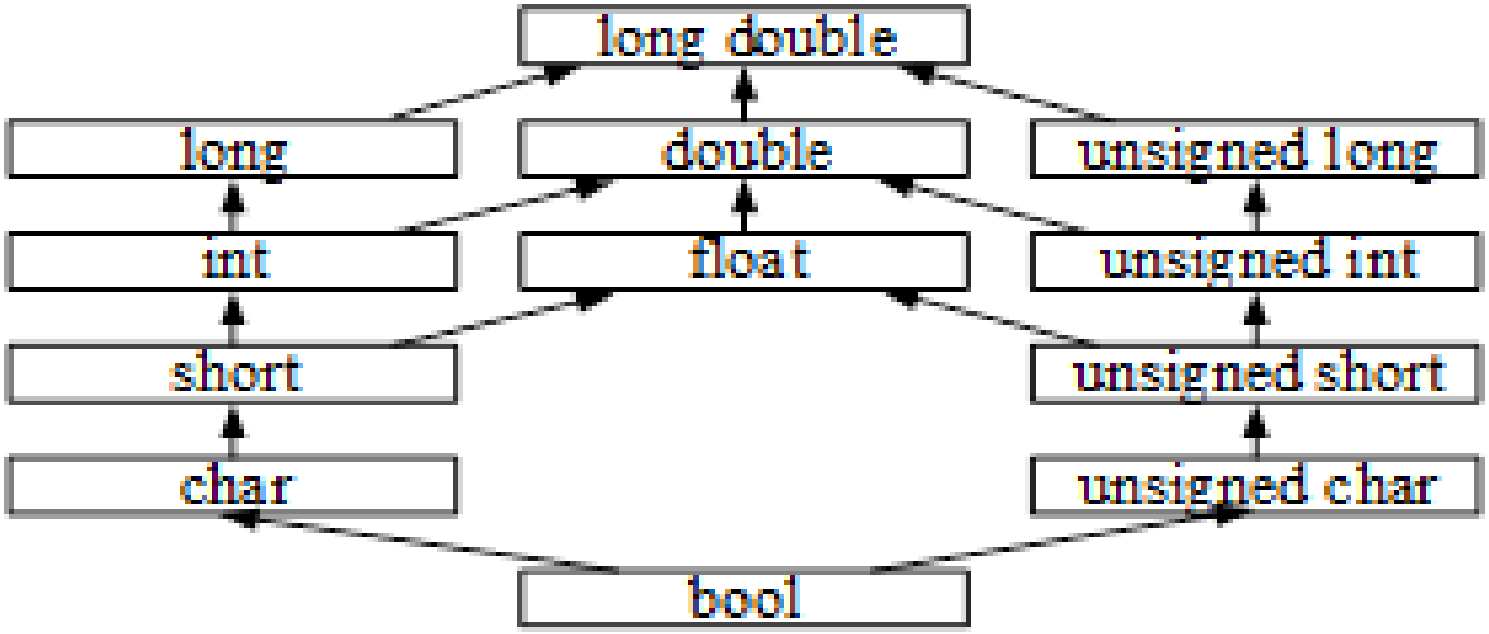
ж) `c=(a=5, b=a*a);` — эквивалентно **a=5; b=a*a; c=b;**

з) `a=(b=s/k)+n;` — эквивалентно **b=s/k; a=b+n;**

и) `c=(a>b)?a:b;` — если a>b, то c=a, иначе c=b

Неявное преобразование типов

При выполнении большинства операций требуется, чтобы операнды были одного типа. Если операнды разных, но совместимых типов, то осуществляется стандартное *неявное преобразование типов*, которое гарантирует *сохранение значимости*.



Без потери значимости компилятор выполняет преобразования типов с меньшим диапазоном представляемых значений к типам с большим диапазоном. Операнды типов char, signed char, unsigned char, short и unsigned short всегда при операциях преобразуются как минимум в тип int.

Неявное преобразование типов

Неявно преобразуются:

- операнды целого и вещественного типов – к вещественному типу,
- операнды с разными интервалами представлений – к типу с большим интервалом.

Если при выполнении присваивания правое значение не умещается в разрядную сетку указанной для него слева переменной, то программа выдает неправильный результат.

Примеры:

```
1)int l, i; double e; float q, x;
   q = i * e / (x + l);
```

При записи в q возможна ошибка переполнения!

double

double

float

```
2) int i = __MAX_INT__;
     i += 1;  /* результат операции i = - 2147483648(!), т.е. получен
совершенно неправильный результат */
```

Явное переопределение типов

Для несовместимых типов следует выполнять *явное приведение типов.* Для этого используют функциональную или каноническую формы (C) или шаблоны функций (C++): static_cast, const_cast, reinterpret_cast и dynamic_cast.

- *Функциональная форма* (только для типов, обозначаемых одним словом): **Имя_типа (Выражение) ,** примеры:

```
int(3.14);
float(2/3);
int('A');
```

- *Каноническая форма*: **(Имя_типа)Выражение ,** примеры:

```
(unsigned long)(x/3+2);
(char)123;
```

- *Шаблон static_cast* : **static_cast <Новый тип>(Переменная),** пример:

```
static_cast<int>(value);
```

В отличие от преобразований Си шаблон static_cast исключает недопустимые преобразования, такие как "число -> адрес".

1.5 Простейший ввод/вывод

1.5.1. Ввод-вывод с помощью функций С

А. Форматный ввод /вывод

Ввод:

```
int scanf(Форматная_строка, Список_адресов_переменных);
        // возвращает количество значений или EOF(-1)
(VS)int scanf_s(Форматная_строка, Список_адресов_переменных_
            с_указанием_размера_буфера_для_символов_и_строк>);
```

Вывод:

```
int printf(Форматная_строка, Список_выражений);
```

где Форматная строка – строковая константа, которая помимо символов содержит спецификации для каждого выводимого значения:

$ Спецификация = %[-] [Целое_1[.Целое_2]] [(h|l|L)] Формат

- - выравнивание по левой границе,

Целое_1 - ширина поля вывода;

Целое_2 - количество цифр дробной части вещественного числа;

h, l, L - модификаторы формата;

Формат – спецификация формата для значения.

Спецификации формата

d,i - целое десятичное число (int);

u - целое десятичное число без знака (unsigned int);

o - целое число в восьмеричной системе счисления;

x,X - целое число в шестнадцатеричной системе счисления, % 4x - без гашения незначащих нулей, X – буквы верхнего регистра;

f - вещественное число float в форме с фиксированной точкой;

e,E - вещественное число float в форме с плавающей точкой;

g,G - вещественное число float в одной из указанных выше форм;

c - символ;

p - указатель (адрес) в шестнадцатеричном виде;

s - символьная строка.

Кроме этого, форматная строка может содержать:

\n - переход на следующую строку;

\n hhh - вставка символа с кодом ANSI hhh (код задается в шестнадцатеричной системе счисления);

%% - печать знака %.

Примеры форматного ввода/вывода

а) `int i=26;`

`printf ("%-6d∪∪∪%%∪ %o∪ %X\n", i, i, i);`

26	∪	∪	∪	∪	∪	∪	%	∪	32	∪	1A ↵

б) `scanf("%d %d",&a,&b);`

Вводимые значения: 1) **24 28** 2) **24**↵
28

в) `scanf("%d,%d",&a,&b);`

Вводимые значения: **24,28**

г) `scanf("%s",name);`

Вводимые значения: **Иванов Иван**

Результат ввода: name="**Иванов**"

Ввод строки до пробела

д) `int i; char ch, name[20];`

`scanf_s("%d %s %c",&i,name,20,&ch,1);`

В параметрах функции указаны размеры буферов

Модификаторы формата

Модификаторы употребляются с некоторыми форматами для указания типов переменных, отсутствовавших в первых версиях Си/С++.

Модификатор	Спецификатор формата	Тип переменной
h	i d u o x X	short
l	i d u o x X	long
l	e E f g G	double
L	e E f g G	long double

Примеры:

```
short s1; long L1; double d1;
printf("input short>"); scanf("%hd", &s1);
printf("input long >"); scanf("%ld", &L1);
printf("input double>"); scanf("%lf", &d1);
```

Б. Ввод/вывод строк

Ввод:

char* gets(Строковая_переменная**);**

// результат: копия строки или **nullptr**

(VS) **char* gets_s**(Строковая_переменная, Размер буфера);

(Clang) **char* fgets**(Строковая_переменная, Размер буфера, **stdin**);

Вывод:

int puts(Строковая константа или переменная);

Примеры:

а) **puts("Это строка");** // Вывод: **Это строка**↵

б) **char st[21]; fgets(st,20,stdin);**

Вводимые значения: **Иванов Иван**↵

Результат: **st ="Иванов Иван**↵**"**

Ввод строки вместе с маркером "конец строки"

в) **char st[21]; gets_s(st,20);** // один байт для хранения '\0'

В. Ввод/вывод символов

Ввод символа:

`int getchar();` // возвращает символ или **EOF**

Вывод символа:

`int putchar(`Символьная_переменная_или_константа`);`

Примеры:

а) `ch=getchar( );`

б) `putchar('t');`

1.5.2 Ввод-вывод с использованием библиотеки классов C++

Операции ввода-вывода с консолью могут осуществляться с использованием специальной библиотеки классов C++.

Для осуществления операций ввода-вывода с консолью необходимо подключить библиотеку **`iostream`**, содержащую описание этих классов, и разрешить программе использовать стандартное пространство имен **`std`**, в котором работают все старые библиотеки C++:

```
#include <iostream>
using namespace std;
```

Вывод на экран

Операция вывода на экран компьютера предполагает вставку данных в стандартный поток вывода.

Стандартный поток вывода на экран

```
cout << Имя_переменной_или_константы;
```

Операция вставки в поток

С помощью операции вставки в поток можно выводить данные следующих типов: **int, short, long, double, char, bool, char * (строки С), void * (адрес) и т.п.**

Примеры вывода на экран

1) вывод строковых констант, чисел и логических значений:

```
int a=3;  float b=5.34; bool c=true;
cout<< "Results: a=" << a << " b=" << b <<
        " c=" << c << "." << '\n';
```

```
Results: a=3 b=5.34 c=1.↵
```

переход на следующую строку, можно также использовать endl

2) вывод в две строки:

```
cout<< "Results: a=" << a << " b=" << b <<'\n'<<
        " c=" << c << "." << '\n';
```

```
Results: a=3 b=5.34↵
c=1.↵
```

Управление выводом. Манипуляторы

Манипуляторы – специальные методы классов ввода-вывода, предназначенные для управления операциями ввода-вывода. Они непосредственно "вставляются" в поток. Манипуляторы бывают с параметрами и без. Например, манипуляторы без параметров:

1) **`scientific`** – устанавливает вывод числа в экспоненциальной ф.;
2) **`left`** – устанавливает вывод по левой границе.

Для использования манипуляторов с параметрами необходимо подключить библиотеку **`iomanip`**: **`#include <iomanip>`**

Например:

1) **`setw(int n)`** – устанавливает ширину поля печати n;
2) **`setprecision(int n)`** - устанавливает количество выводимых цифр числа.

Примеры:

```
int a=3; double b=-5.543;
cout << setw(8) << a << endl;
cout << setw(8) << setprecision(2) << b << endl;
```

```
       3↵
    -5.5↵
```

Ввод с клавиатуры

Операция ввода с клавиатуры программируется как операция извлечения из потока.

Стандартный поток ввода с клавиатуры

```
cin >> Имя_скалярной_переменной_или_строки;
```

Операция извлечения из потока

Можно вводить целые и вещественные числа, символы, строки, булевские значения: int, long, double, char, char *(строки) и т.д.

Значения (кроме символов) следует разделять пробелами и/или маркерами перехода на следующую строку. Символы при вводе не разделяются. Строки вводятся до пробела!

Примеры ввода с клавиатуры

1) ввод чисел:

```
int a;  float b; bool c;
cout << "Enter a, b, c: ";
cin >> a >> b >> c;
```

```
Enter a, b, c: 3 5.1 true↵
```

```
Enter a, b, c: ↵
3↵
5.1↵
true↵
```

2) ввод символов:

```
char ch1,ch2;
cout<< "Enter ch1, ch2: ";
cin >> ch1 >> ch2;
```

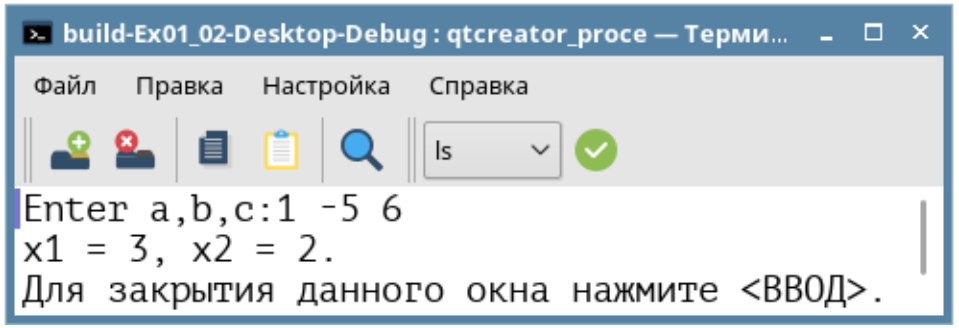
```
Enter ch1, ch2: ab↵
```

3) пропуск Enter перед вводом символов и строк:

```
cin >> n;
cin.ignore(2,'\n');
cin >> str1;
```

```
Enter n: 4↵
abcd
```

Программа определения корней кв. уравнения (только для действительных корней)

Ex01_02

```
#include <iostream>
#include <cmath>
using namespace std;
int main()
{
    float a,b,c,d,e,x1,x2;
    cout << "Enter a,b,c:";
    cin >> a >> b >> c;
    d = sqrt(b * b - 4 * a * c);
    e = 2 * a;
    x1 = (-b + d) / e;
    x2 = (-b - d) / e;
    cout << "x1 = " << x1 << ", x2 = " << x2 << "." << endl;
    return 0;
}
```